GENERALITAT VALENCIANA
Conselleria d'Educació, Investigació, Cultura i Esport

Institut Educació Secundària El Caminàs

Unió Europea
Fons Social Europeu
El FSE inverteix en el teu futur

Ciclo Formativo Grado Superior: DAW
Curso: 2º



Forgejo detrás de Nginx

Autor Original: Juan Carlos Requena
Adaptación: Miguel Albert Soler
Modificación: Ivan Rubert Albiol

Objetivos

- Completar la guía paso a paso para instalar y configurar forgejo con nginx en docker

Resumen

Noviembre-2026

Dpto. Informática - IES El Caminás

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0



Revisiones

Revisión	Fecha	Descripción
1.0.	07-11-2026	Guia paso a paso forgejo con nginx en docker

Índice de Contenidos

1. Introducción 3
2. Preparación.....4
3. Creación de los contenedores con Docker Compose.....4
4. Primera configuración de Forgejo 5
5. Despliegue de Forgejo con proxy inverso Nginx 8
6. Probando subir un archivo grande (fallo provocado).....11
7. Desplegar segundo sitio en el mismo servidor 13
8. Comprobación de la pagina pages.....14
9. Nombre de dominio que no existe.....15
10. Leer HTTP en crudo 15
11. Añadir HTTPS.....17
12. Git con HTTPS.....21
13. Comprobaciones finales 21
14. Demostración final 22
15. Bibliografía y otras fuentes de información consultadas 29

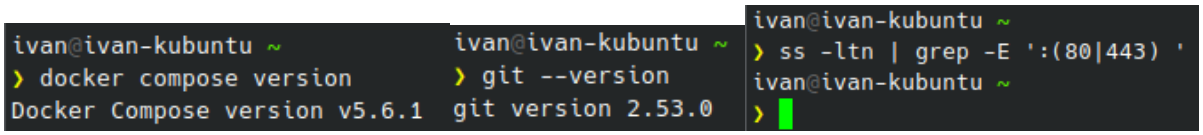
1. Introducción

Se va a crear un contenedor docker que contendrá Forgejo local desplegado detrás de Nginx, con dos sitios en el mismo servidor y con HTTPS.

Para este proyecto se necesita Docker, Docker Compose, una terminal, un navegador (en esta guia se usará Brave) y git

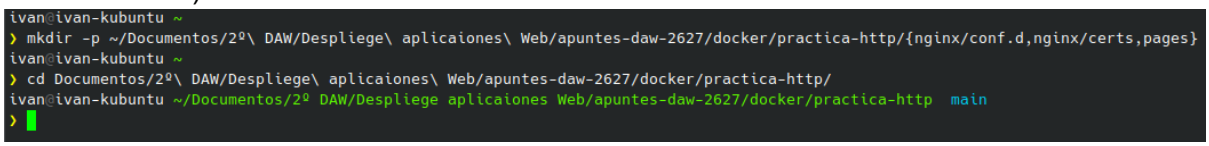
2. Preparación

Primero se revisa si se tiene instalado Docker, docker compose, git y que los puertos 80 (HTTP) y 443 (HTTPS) no están en uso



Docker y Docker Compose *Git* *Puertos HTTP y HTTPS*

A continuación se crea la carpeta del proyecto (en la practica lo sitúa en la carpeta home del usuario pero en mi caso se creará en otra ruta para tener los proyectos organizados correctamente)



3. Creación de los contenedores con Docker Compose

Se crea el Docker Compose en la carpeta raíz del proyecto con el siguiente contenido para configurar inicialmente Forgejo y tener las carpetas de los bind mount de ambos contenedores antes de modificar el docker-compose.yml para añadir Nginx como proxy inverso

Forgejo detras de Nginx

docker-compose.yml

```yaml
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: forgejo
      POSTGRES_PASSWORD: forgejo
      POSTGRES_DB: forgejo
    volumes:
      - ./postgres:/var/lib/postgresql/data

  forgejo:
    image: codeberg.org/forgejo/forgejo:16
    restart: unless-stopped
    depends_on:
      - db
    environment:
      USER_UID: "1000"
      USER_GID: "1000"
      FORGEJO__database__DB_TYPE: postgres
      FORGEJO__database__HOST: db:5432
      FORGEJO__database__NAME: forgejo
      FORGEJO__database__USER: forgejo
      FORGEJO__database__PASSWD: forgejo
      FORGEJO__server__DISABLE_SSH: "true"
      FORGEJO__server__ROOT_URL: http://localhost:3000/
    volumes:
      - ./forgejo:/data
    ports:
      - "3000:3000"
```

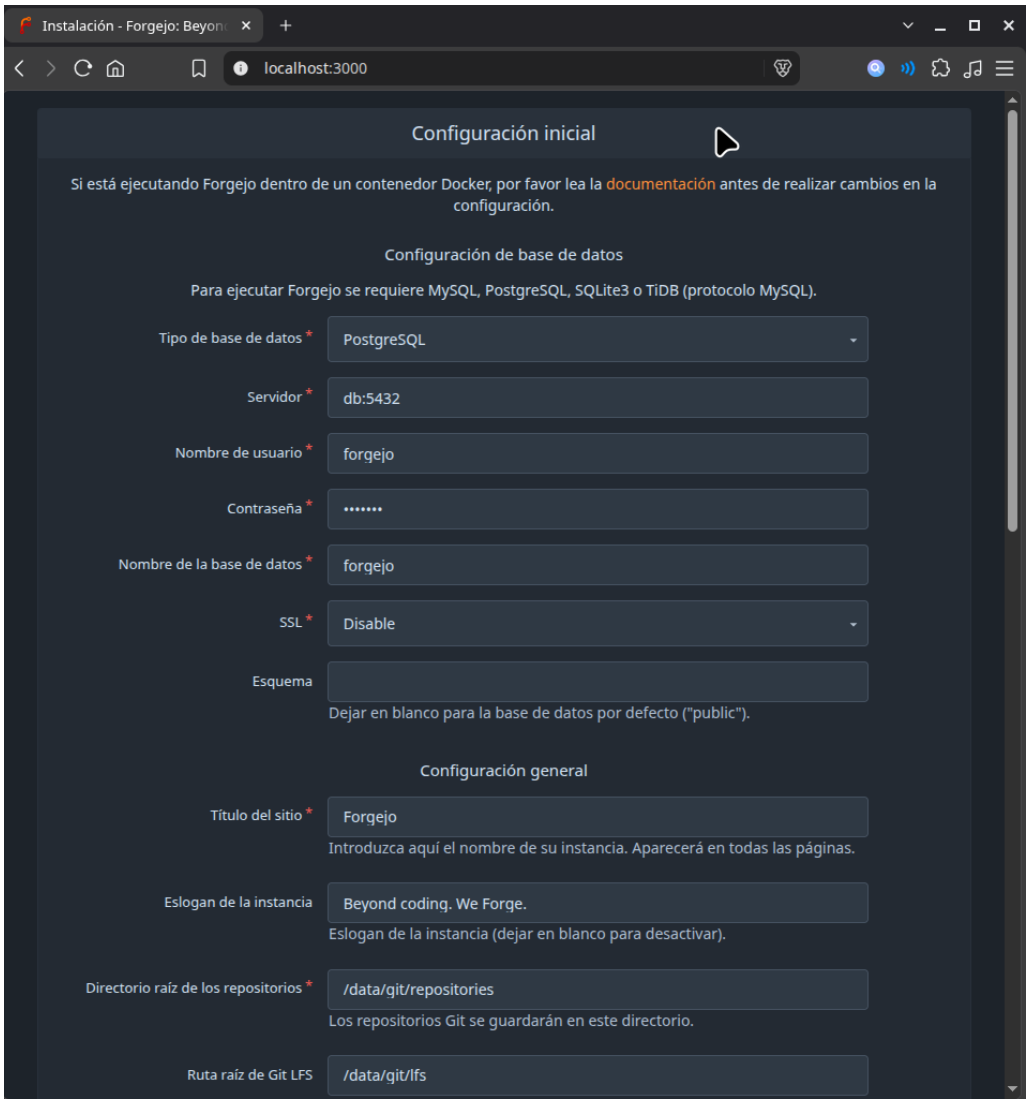
Lanzamos los contenedores y esperamos a que Forgejo se despliegue y empiece a escuchar por el puerto 3000 que hemos configurado para seguir con el siguiente punto de primera configuración de Forgejo

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose up -d
[+] up 29/29
 ✔ Image codeberg.org/forgejo/forgejo:16 Pulled                                  11.3s
 ✔ Image postgres:16-alpine              Pulled                                  14.0s
 ✔ Network practica-http_default         Created                                  0.1s
 ✔ Container practica-http-db-1          Started                                  1.1s
 ✔ Container practica-http-forgejo-1     Started                                  0.7s
```

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose logs -f forgejo
forgejo-1  | Generating /data/ssh/ssh_host_ed25519_key...
forgejo-1  | Generating /data/ssh/ssh_host_rsa_key...
forgejo-1  | 2026/11/06 16:16:59 ...nvironment-to-ini.go:104:runEnvironmentToIni() [I] Settings saved to: "/data/gitea/conf/app.ini
"
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:250:runWeb() [I] Starting Forgejo on PID: 16
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:114:showWebStartupMessage() [I] Forgejo version: 16.0.7+gitea-1.22.0 built with GNU Mak
e 4.4.1, go1.26.10 : bindata, timetzdata, sqlite, sqlite_unlock_notify
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:115:showWebStartupMessage() [I] * RunMode: prod
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:116:showWebStartupMessage() [I] * AppPath: /usr/local/bin/gitea
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:117:showWebStartupMessage() [I] * WorkPath: /data/gitea
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:118:showWebStartupMessage() [I] * CustomPath: /data/gitea
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:119:showWebStartupMessage() [I] * ConfigFile: /data/gitea/conf/app.ini
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:120:showWebStartupMessage() [I] Prepare to run install page
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:315:listen() [I] Listen: http://0.0.0.0:3000
forgejo-1  | 2026/11/06 16:17:00 cmd/web.go:319:listen() [I] AppURL(ROOT_URL): http://localhost:3000/
forgejo-1  | 2026/11/06 16:17:00 ...s/graceful/server.go:50:NewServer() [I] Starting new Web server: tcp:0.0.0.0:3000 on PID: 16
forgejo-1  | Generating /data/ssh/ssh_host_ecdsa_key...
forgejo-1  | Server listening on :: port 22.
forgejo-1  | Server listening on 0.0.0.0 port 22.
```

4. Primera configuración de Forgejo

Se accede a http://localhost:3000, bajamos hasta la opción de "*Configuración de la cuenta de administrador*", se rellenan los campos y se pulsa en "*Instalar Forgejo*"



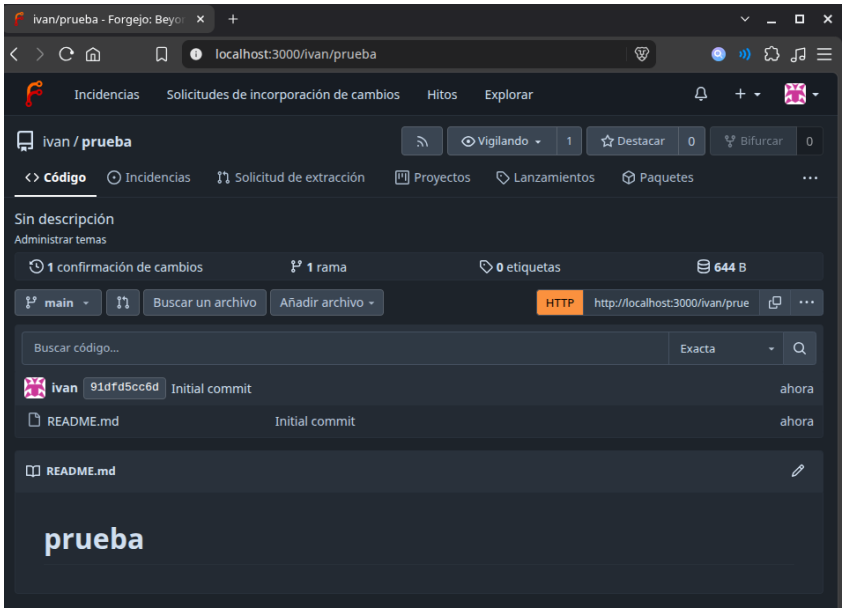


Una vez instalado nos redirigirá a la pantalla principal de nuestro usuario administrador, si no inicia sesión se debe ejecutar el siguiente comando en la terminal

```
1. docker compose exec -u 1000 forgejo forgejo admin user create --admin \
2.   --username tu_usuario --password 'Cambiar1234!' --email
   tu_usuario@example.com \
3.   --must-change-password=false
```

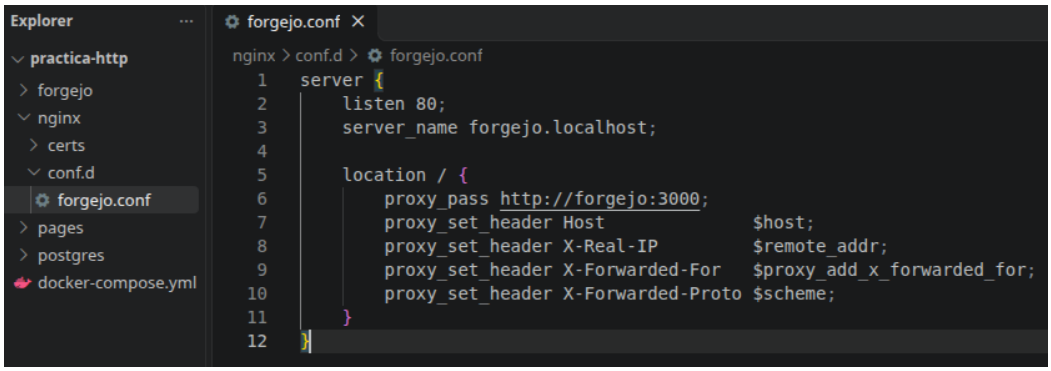
Para comprobar el correcto funcionamiento creamos un repositorio de prueba con un README y se comprueba que se vea





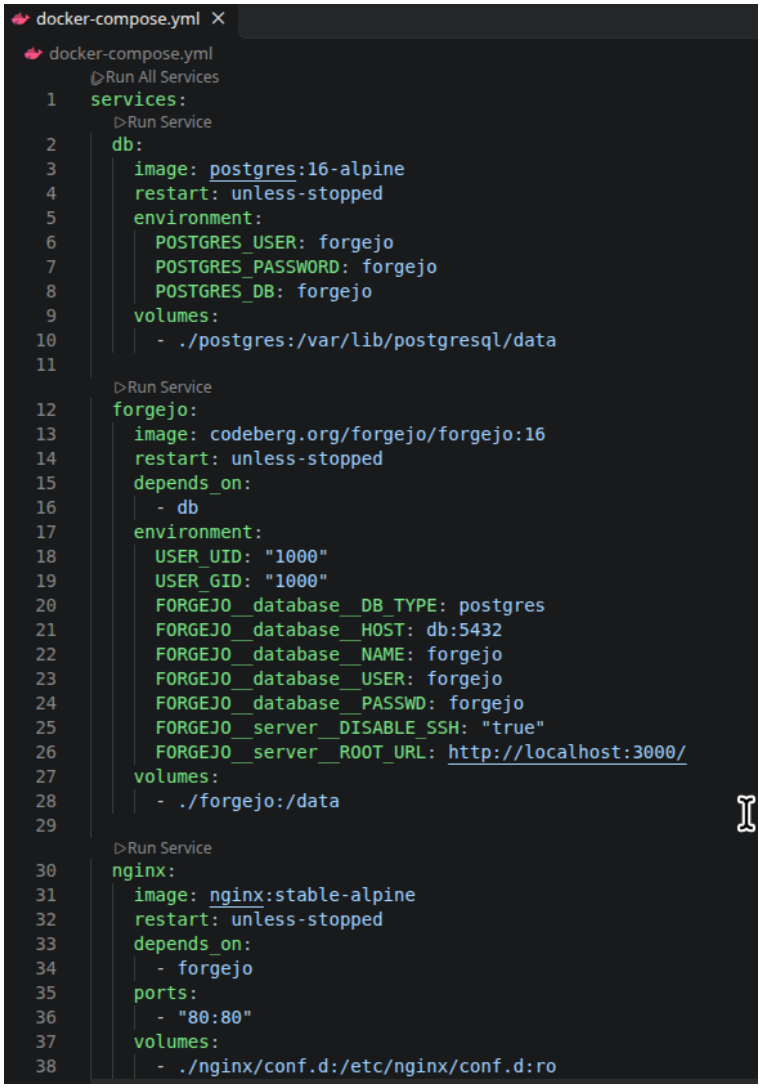
5. Despliegue de Forgejo con proxy inverso Nginx

Ahora que ya se ha hecho la configuración inicial y comprobado que funciona creamos un archivo de configuración de un sitio web para Forgejo en la carpeta nginx/conf.d de la raiz del proyecto con el siguiente contenido



Después de crear el archivo se modifica el docker-compose.yml para añadir Nginx y se elimina el puerto expuesto del mismo archivo ya que lo servirá a través del proxy inverso por el puerto 80

Forgejo detras de Nginx



Se arranca el contenedor de Nginx y se comprueba que con el dominio http://forgejo.localhost llega a forgejo mientras que http://localhost:3000 ya no funciona

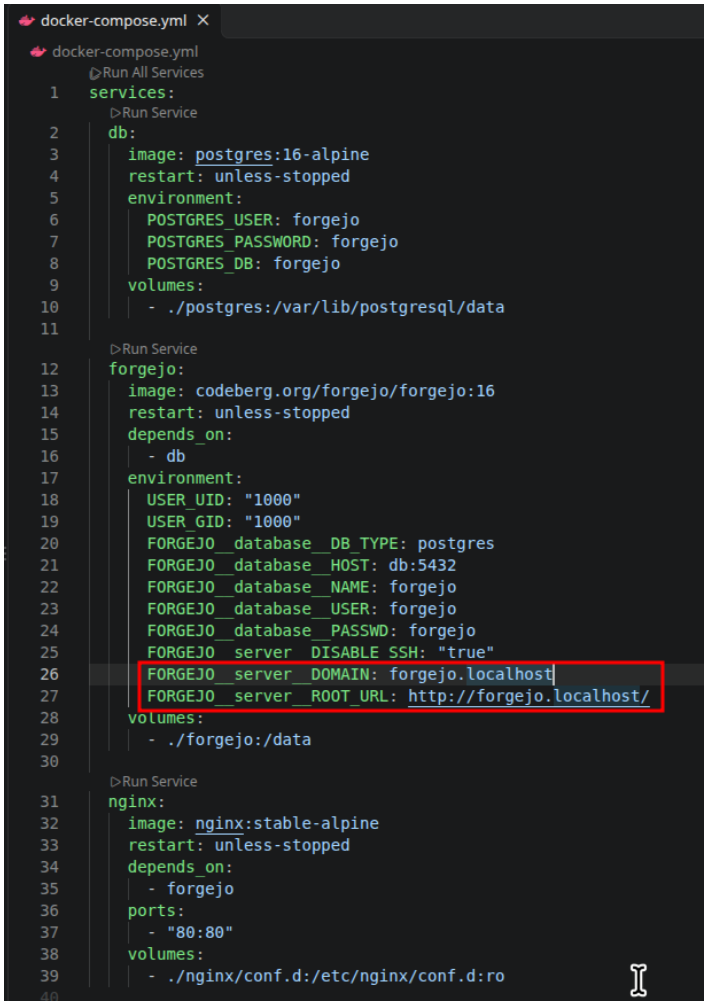
Forgejo detras de Nginx

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose up -d
[+] up 14/14
 ✔ Image nginx:stable-alpine         Pulled                                                                3.4s
 ✔ Container practica-http-db-1      Running                                                               0.0s
 ✔ Container practica-http-forgejo-1 Started                                                               1.9s
 ✔ Container practica-http-nginx-1   Started                                                               0.8s
```




```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -I http://localhost:3000
curl: (7) Failed to connect to localhost port 3000 after 0 ms: Could not connect to server
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -I http://forgejo.localhost
HTTP/1.1 200 OK
Server: nginx/1.30.6
Date: Fri, 06 Nov 2026 16:53:37 GMT
Connection: keep-alive
```

Pero ahora con el proxy inverso podrían haber ubicaciones dentro de Forgejo que no carguen bien para eso volvemos a modificar el docker-compose.yml añadiendo la siguiente línea y modificando ROOT_URL y volvemos a lanzar los contenedores



```yaml
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: forgejo
      POSTGRES_PASSWORD: forgejo
      POSTGRES_DB: forgejo
    volumes:
      - ./postgres:/var/lib/postgresql/data

  forgejo:
    image: codeberg.org/forgejo/forgejo:16
    restart: unless-stopped
    depends_on:
      - db
    environment:
      USER_UID: "1000"
      USER_GID: "1000"
      FORGEJO__database__DB_TYPE: postgres
      FORGEJO__database__HOST: db:5432
      FORGEJO__database__NAME: forgejo
      FORGEJO__database__USER: forgejo
      FORGEJO__database__PASSWD: forgejo
      FORGEJO__server__DISABLE_SSH: "true"
      FORGEJO__server__DOMAIN: forgejo.localhost
      FORGEJO__server__ROOT_URL: http://forgejo.localhost/
    volumes:
      - ./forgejo:/data

  nginx:
    image: nginx:stable-alpine
    restart: unless-stopped
    depends_on:
      - forgejo
    ports:
      - "80:80"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
```



```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose up -d && docker compose restart nginx
[+] up 3/3
 ✔ Container practica-http-nginx-1   Running                                    0.0s
 ✔ Container practica-http-db-1      Running                                    0.0s
 ✔ Container practica-http-forgejo-1 Started                                    1.6s
[+] restart 0/1
 ⠸ Container practica-http-nginx-1 Restarting                                   0.6s
```

6. Probando subir un archivo grande (fallo provocado)

Se intenta subir un archivo al repositorio clonado pero falla al hacer el git push con un error 413 (Request Entity Too Large)

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ cd ~
ivan@ivan-kubuntu ~
❯ git clone http://forgejo.localhost/ivan/prueba.git
Clonando en 'prueba'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Recibiendo objetos: 100% (3/3), listo.
ivan@ivan-kubuntu ~
❯ cd prueba
ivan@ivan-kubuntu ~/prueba  main
❯ dd if=/dev/urandom of=grande.bin bs=1M count=5
5+0 registros en
5+0 registros fuera
5242880 bytes (5.2 MB, 5.0 MiB) copiados, 0.0239178 s, 228 MB/s
ivan@ivan-kubuntu ~/prueba  main
❯ git add grande.bin
ivan@ivan-kubuntu ~/prueba  main●
❯ git commit "Fichero grande"
error: ruta especificada 'Fichero grande' no concordó con ningún archivo conocido por git
ivan@ivan-kubuntu ~/prueba  main●
❯ git commit -m "Fichero grande"
[main d90749c] Fichero grande
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 grande.bin
ivan@ivan-kubuntu ~/prueba  main
❯ git push
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (3/3), listo.
error: RPC falló; HTTP 413 curl 22 The requested URL returned error: 413
send-pack: unexpected disconnect while reading sideband packet
Escribiendo objetos: 100% (3/3), 5.00 MiB | 7.76 MiB/s, listo.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
fatal: el remoto se colgó de manera inesperada
Everything up-to-date
ivan@ivan-kubuntu ~/prueba  main
❯
```

Revisamos los logs de nginx buscando ese error

```
ivan@ivan-kubuntu ~/prueba  main
❯ cd ~/Documentos/2º\ DAW/Despliege\ aplicaiones\ Web/apuntes-daw-2627/docker/practica-http/
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose logs nginx | grep "too large"
nginx-1  | 2026/11/06 17:10:23 [error] 20#20: *73 client intended to send too large chunked body: 1032371+65515 bytes, client: 172.
18.0.1, server: forgejo.localhost, request: "POST /ivan/prueba.git/git-receive-pack HTTP/1.1", host: "forgejo.localhost"
```

Nginx no permite las peticiones con un body superior a 1MB por defecto por lo que se debe modificar el archivo de configuración para permitir la petición de hasta 512MB y recargamos nginx sin pararlo (primero testeamos los archivos “-t” y luego recargamos el servicio “-s reload”)

forgejo.conf

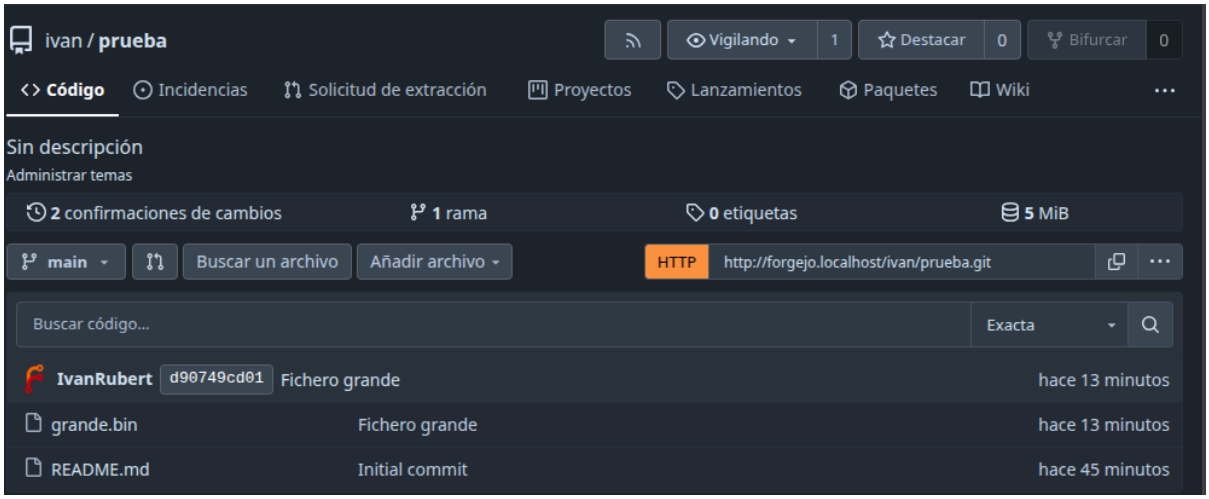
nginx > conf.d > forgejo.conf

```
1  server {
2      listen 80;
3      server_name forgejo.localhost;
4      client_max_body_size 512m;
5
6      location / {
7          proxy_pass http://forgejo:3000;
8          proxy_set_header Host              $host;
9          proxy_set_header X-Real-IP         $remote_addr;
10         proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
11         proxy_set_header X-Forwarded-Proto $scheme;
12     }
13 }
```

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose exec nginx nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose exec nginx nginx -s reload
2026/11/06 17:19:49 [notice] 30#30: signal process started
```

Una vez modificada la configuración volvemos a ~/prueba y volvemos a hacer el push y revisamos el repositorio de forgejo

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ cd ~/prueba/
ivan@ivan-kubuntu ~/prueba  main
❯ git push
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 5.00 MiB | 7.59 MiB/s, listo.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://forgejo.localhost/ivan/prueba.git
   91dfd5c..d90749c  main -> main
ivan@ivan-kubuntu ~/prueba  main
❯
```

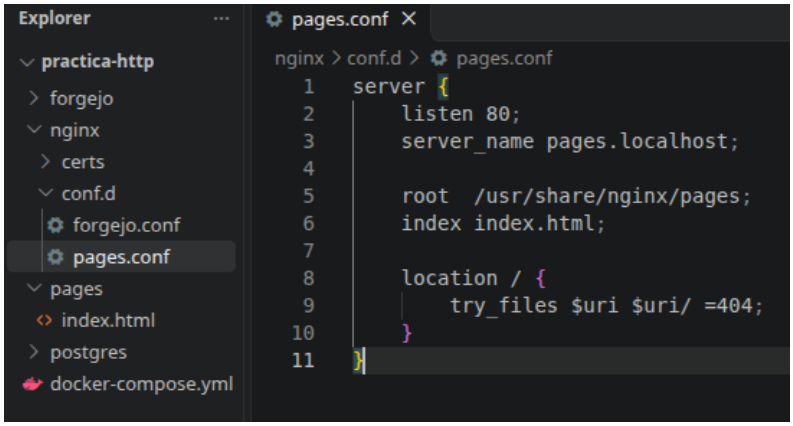


7. Desplegar segundo sitio en el mismo servidor

En el equipo anfitrión creamos el archivo index.html en la ruta pages de la raíz del proyecto

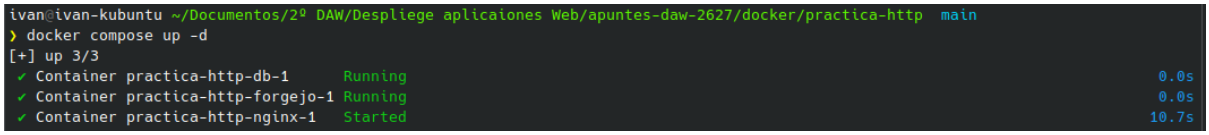


Una vez creado creamos el archivo de configuración del segundo sitio en nginx/conf.d



Ya lo tenemos preparado, pero nos falta pasar el HTML al contenedor, para eso modificamos el docker-compose.yml añadiendo el bind mount de la carpeta pages como solo lectura y volvemos a lanzarlo





8. Comprobación de la pagina pages

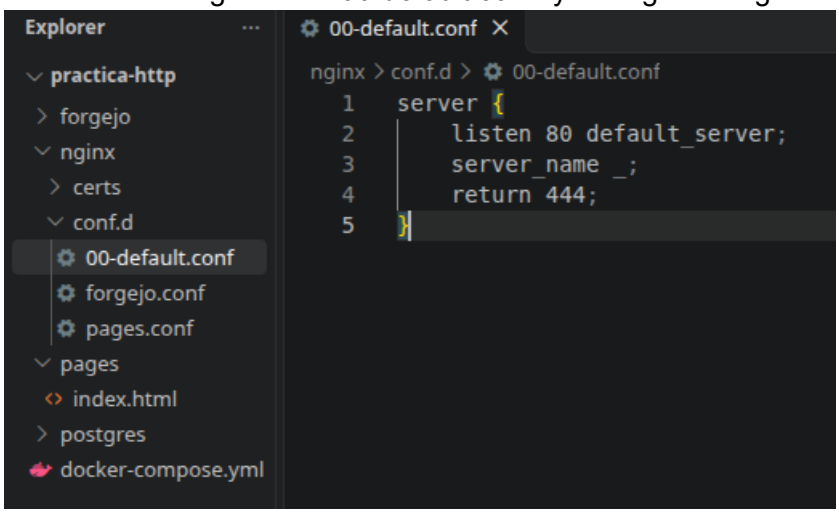
Una vez ya desplegado el contenedor modificado probamos el dominio http://pages.localhost con el navegador y curl




```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -I http://pages.localhost
HTTP/1.1 200 OK
Server: nginx/1.30.6
Date: Fri, 06 Nov 2026 17:37:38 GMT
Content-Type: text/html
Content-Length: 234
Last-Modified: Fri, 06 Nov 2026 17:26:03 GMT
Connection: keep-alive
ETag: "6aee0dab-ea"
Accept-Ranges: bytes
```

9. Nombre de dominio que no existe

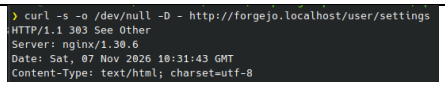
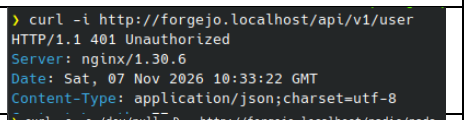
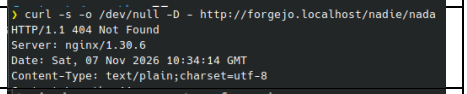
Cuando se busca un nombre de dominio que no existe en la configuración de Nginx se dirige a la primera entrada que coincida con el puerto por defecto, así que crearemos el archivo de configuración "*00-default.conf*" y recargamos nginx antes de volver a probarlo



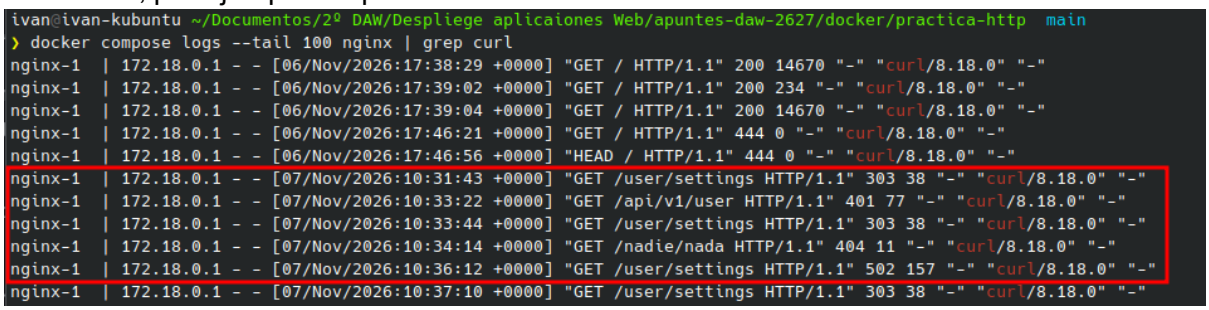

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose exec nginx nginx -t && docker compose exec nginx nginx -s reload
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/11/06 17:45:52 [notice] 29#29: signal process started
```

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -v -H "Host: hola.localhost" http://127.0.0.1/
*   Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 48450
* using HTTP/1.x
> GET / HTTP/1.1
> Host: hola.localhost
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* Empty reply from server
* shutting down connection #0
curl: (52) Empty reply from server
```

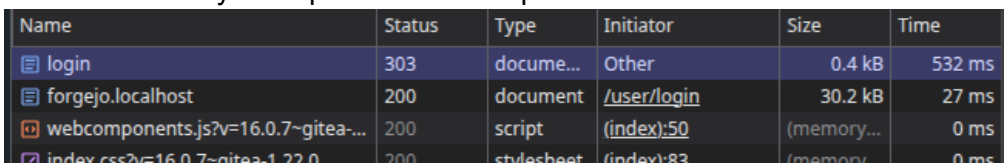
10. Leer HTTP en crudo

Código	Comando	Resultado
303 (Redireccion)	curl -s -o /dev/null -D - http://forgejo.localhost/user/settings	
401 (No autenticado)	curl -i http://forgejo.localhost/api/v1/user	
404 (Not encontrado)	curl -s -o /dev/null -D - http://forgejo.localhost/nadie/nada	
502 (Aplicación caída)	docker compose stop forgejo curl -s -o /dev/null -D - http://forgejo.localhost/user/settings	

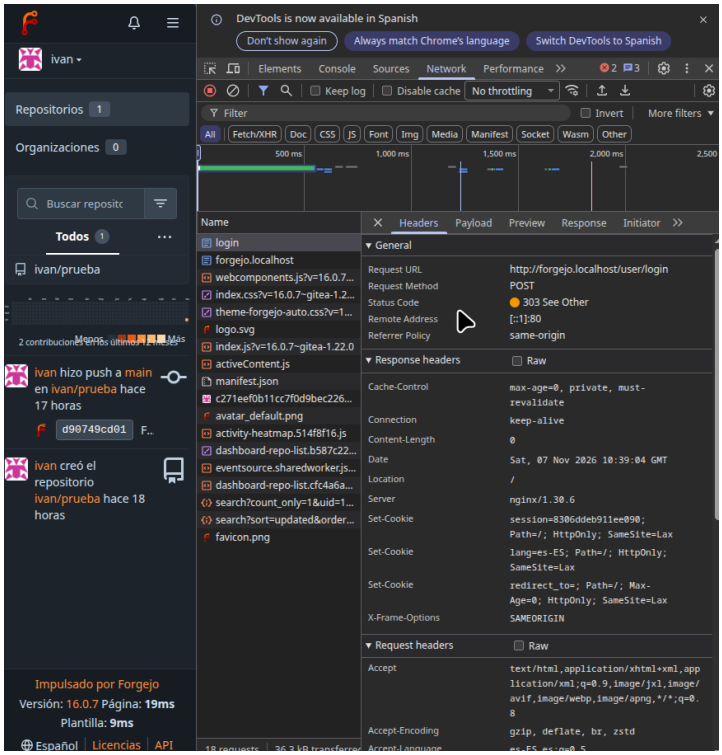
En los logs de Nginx cada petición que se hace queda registrada y se pueden buscar fácilmente, por ejemplo las peticiones hechas anteriormente con curl



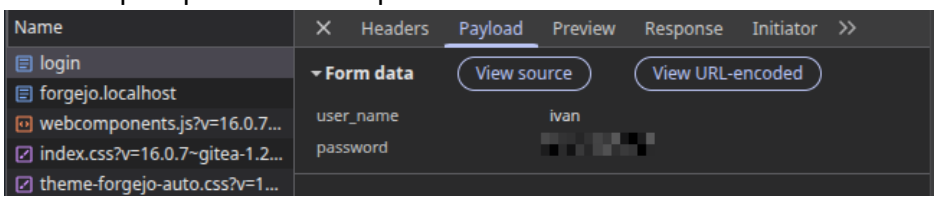
Se inicia sesión y se captura el método post



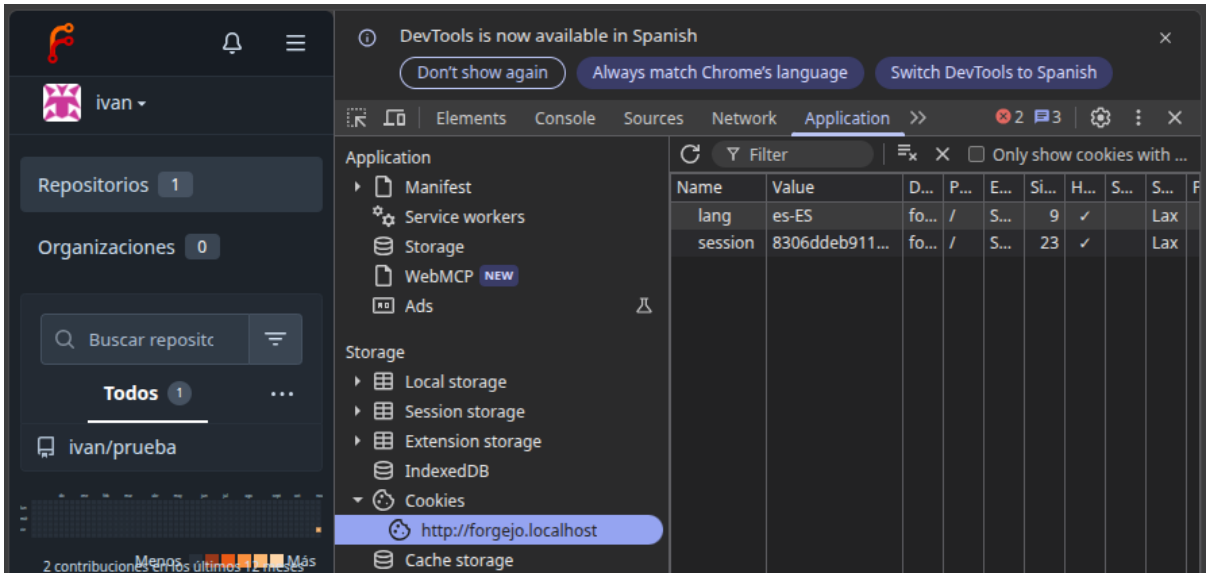
Name	Status	Type	Initiator	Size	Time
login	303	docume...	Other	0.4 kB	532 ms
forgejo.localhost	200	document	/user/login	30.2 kB	27 ms
webcomponents.js?v=16.0.7~gitea-...	200	script	(index):50	(memory...	0 ms
index.css?v=16.0.7~gitea-1.22.0	200	stylesheet	(index):83	(memory...	0 ms



Al ser http se pueden interceptar las credenciales



Ahora comprobamos la cookie de sesión en Application > Cookies



En la sesión hay estos 3 valores que indican que estamos en una conexión HTTP permitiendo el envio de la cookie en peticiones GET, pero bloquea la cookie en peticiones POST y requests XHR

HttpOnly	SameSite	Secure
✓	Lax	

11. Añadir HTTPS

Primero se debe crear un certificado de una unidad certificadora. En nuestro caso crearemos una clave privada y un certificado autofirmados sin contraseña (-noenc) y valido durante 5 años (-days 1825)

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ cd nginx/certs/
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http/nginx/certs  main
❯ openssl req -x509 -newkey rsa:4096 -noenc -keyout ca.key -out ca.crt -days 1825 -subj "/CN=CA DAW Ivan"
.+...+.....+.+.....+.+...+..+.+........+..............+.+...+...+.....+...+....+..+............+..+...+.+.....................
...+..+.....+.+.....+....+............+..+.+......++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+..+.....+.....+.+..+.........
+.....+..............+..........+.+..+...+++++++++++++++++++++++++++++++++++++++++++++*....+.....+...+...+.+.......+...+...+...
......+.......+......+...+.....+...+........+.......+.....................+..+.......+......+.............+.+..+.........+.......
...+.....+...+..+...+..............+.+.....+..............+...+...+.......+.....+...+.....................+......+.......
..+..+.....................+.............+......+.+.........+...............+.....+......+...+.+..+.....+.........+...+......
.+.....+......+.+...+..............+...+...+..+.........................+...........+.+...+......+.....+.+..+...........+.....
....+.......................+......+...+..+.........+..............+.......+..............+........+......+.........+.........
...........+..+...+..............+....+.........+......+........+......+.....+++++
......+.+.....++++++++++++++++++++++++++++++++++++++++++++++*...........+.......+..+..........+..+......+...++++++++++++++++++++++
++++++++++++++++++++*......+.+.....+++++
-----
```

A continuación, se crea un certificado para el servidor (la clave del servidor y una petición de firma)

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http/nginx/certs  main
❯ openssl req -newkey rsa:2048 -noenc -keyout daw.key -out daw.csr -subj "/CN=forgejo.localhost"
.+.++++++++++++++++++++++++++++++++++++++++*.....+......++++++++++++++++++++++++++++++++++++++++*...+....+...+........+.........+....
..+.+......+..+.+..+....+......+..+...+.........+....+..+.........+..................+.+..+...................+...+..............
.+....+..+....+......+.....+.......+...+...+..........+...+.+.........+..+...+......+.......+...+..+.....+.....+.....+......+..
........+...+..+...+...+....+...+..+.+..............+...+..+..............+.......+........++++++
..............+......++++++++++++++++++++++++++++++++++++++++*..+......+......+..+.+......+......+...++++++++++++++++++++++++++++++
++++++++++*.........+...+...+..+..............+..........+.....+...+....+.....+....+............+...+..........+......+...+.....
...+..........+...+..+......+.+......+.....+.......+.......+.....+..+....+.........+..............+..+...+.......+...+.....+.+.
...........+...+............+.....+....+......+..+............+...............+.+...+...........+...+.+..+..............+....+...+
.....+...............+.....+.+.....+.+..+......+.................+..+......+...+...........+............+...+...+....+...+.....
............+.+.....+.........+...+.......+..+......+............+.+..+....+...+..+...+.............+..............+..+.+.....+.+.
....+.+........+.+......+........+..............+.......+..+.......+.............+.+..............+.....+.+........+.+........+.
+...........+.+.....+...+..............+.........+...+.......++++++
-----
```

Y se crea un nombre SAN

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http/nginx/certs  main
❯ cat > san.ext <<EOF
subjectAltName=DNS:forgejo.localhost,DNS:pages.localhost
basicConstraints=CA:FALSE
keyUsage=digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
EOF
```

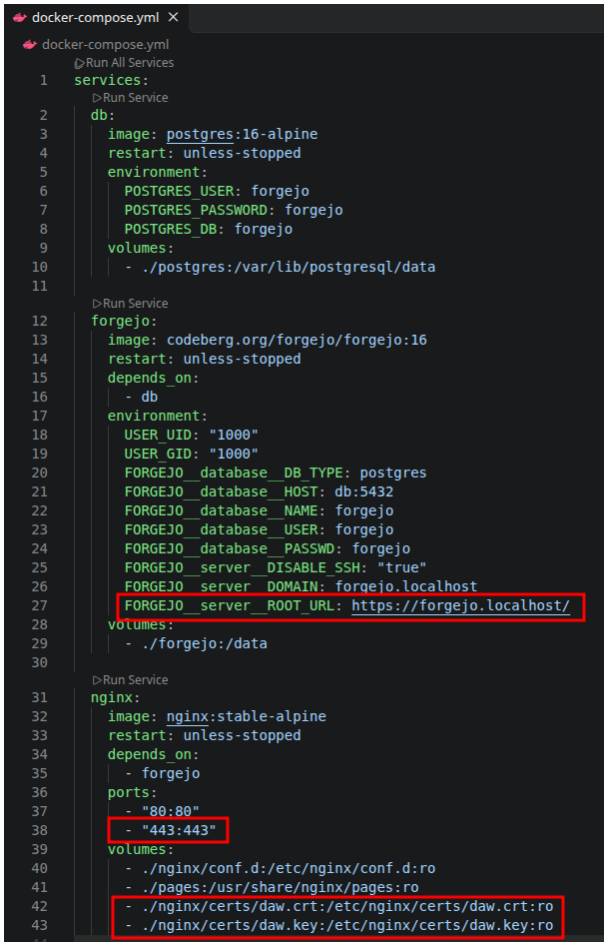
Cuando ya está todo creado, pasamos a firmar con nuestro CA y revisamos que está bien creado

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http/nginx/certs  main
❯ openssl x509 -req -in daw.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out daw.crt -days 825 -sha256 -extfile san.ext
Certificate request self-signature ok
subject=CN=forgejo.localhost
```

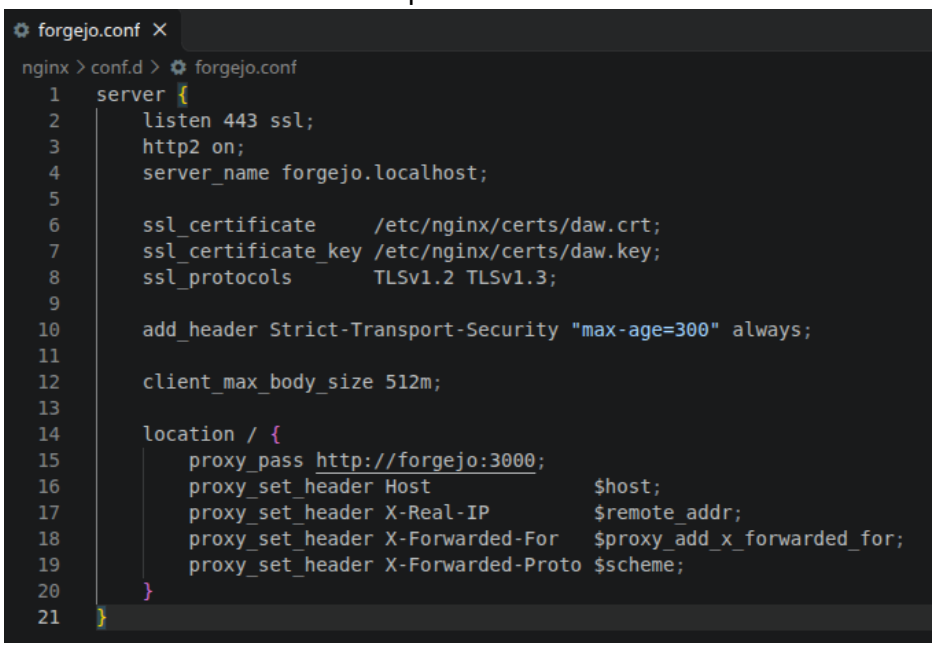
```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http/nginx/certs  main
❯ openssl x509 -in daw.crt -noout -text | grep -A1 "Alternative Name"
            X509v3 Subject Alternative Name:
                DNS:forgejo.localhost, DNS:pages.localhost
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http/nginx/certs  main
❯ openssl verify -CAfile ca.crt daw.crt
daw.crt: OK
```

Una vez ya creado el certificado modificamos el docker-compose.yml añadiendo el puerto 443 y montar el certificado y la clave del servidor

Forgejo detras de Nginx



Y a continuación, modificamos los archivos de configuración de Nginx de fogejo y pages y añadimos la redirección de los puertos 80



pages.conf

```
nginx > conf.d > pages.conf
server {
    listen 443 ssl;
    http2 on;
    server_name pages.localhost;

    ssl_certificate     /etc/nginx/certs/daw.crt;
    ssl_certificate_key /etc/nginx/certs/daw.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    add_header Strict-Transport-Security "max-age=300" always;

    root  /usr/share/nginx/pages;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Explorer

```
practica-http
  forgejo
  nginx
    certs
    conf.d
      00-default.conf
      01-redirect.conf
      forgejo.conf
      pages.conf
```

01-redirect.conf

```
nginx > conf.d > 01-redirect.conf
server {
    listen 80;
    server_name forgejo.localhost pages.localhost;
    return 301 https://$host$request_uri;
}
```

Y por ultimo se añade un servidor por defecto para el 443

00-default.conf

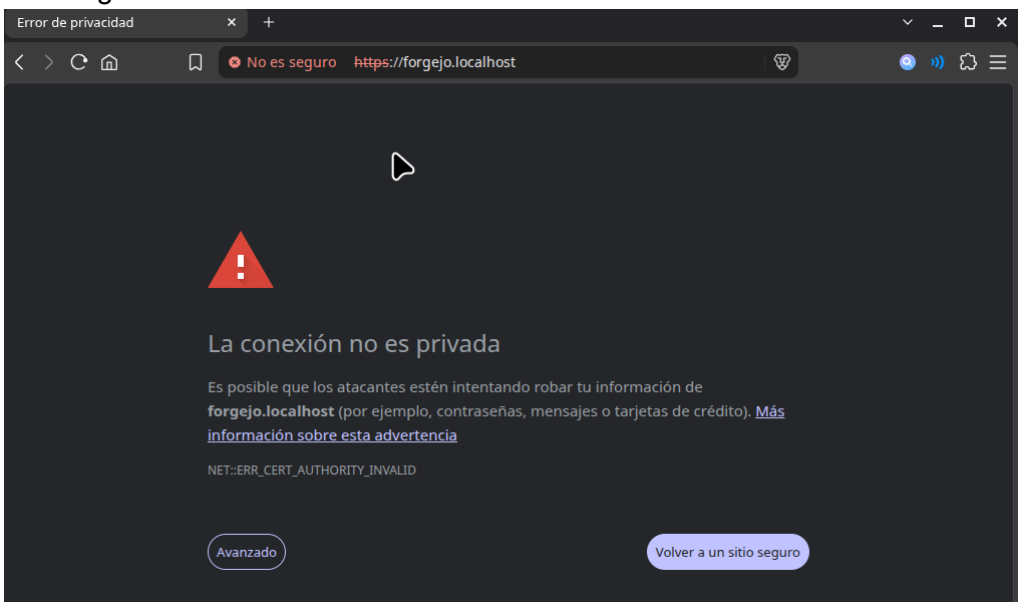
```
nginx > conf.d > 00-default.conf
server {
    listen 80 default_server;
    server_name _;
    return 444;
}

server {
    listen 443 ssl default_server;
    server_name _;
    ssl_reject_handshake on;
}
```

Lo cargamos todo al contenedor y revisamos en el navegador

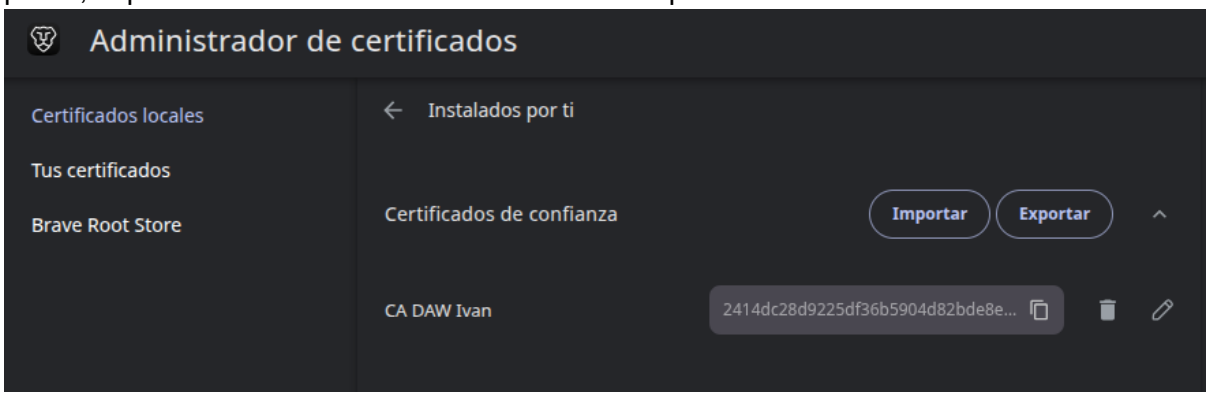
```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose up -d
[+] up 3/3
 ✔ Container practica-http-db-1      Running                                                                0.0s
 ✔ Container practica-http-forgejo-1 Started                                                                2.0s
 ✔ Container practica-http-nginx-1   Started                                                                1.2s
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose exec nginx nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ docker compose restart nginx
[+] restart 0/1
 ⠋ Container practica-http-nginx-1 Restarting                                                               0.5s
```

Como es un certificado autofirmado el navegador no lo reconoce como un sitio seguro y nos da la siguiente advertencia



Para que el navegador confie entramos en Configuración > Privacidad y Seguridad > Seguridad > Gestionar certificados

Se abrirá una nueva pestaña con el Administrador de certificados, pulsamos en “Instalados por ti”, importamos un certificado de confianza e importamos el certificado llamado ca.crt



Y volvemos a intentar entrar en forgejo y veremos que ya no nos sale el mensaje de error

12. Git con HTTPS

Git no confía en nuestro certificado autofirmado por lo que suelta un mensaje de error

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ cd ~/prueba/
ivan@ivan-kubuntu ~/prueba  main
❯ git remote set-url origin https://forgejo.localhost/ivan/prueba.git
ivan@ivan-kubuntu ~/prueba  main
❯ git pull
fatal: no es posible acceder a 'https://forgejo.localhost/ivan/prueba.git/': SSL certificate verification failed: certificate signe
r not trusted. (CAfile: /etc/ssl/certs/ca-certificates.crt CRLfile: none)
```

Lo que haremos es decirle a git que confie en el certificado solo para esté servidor

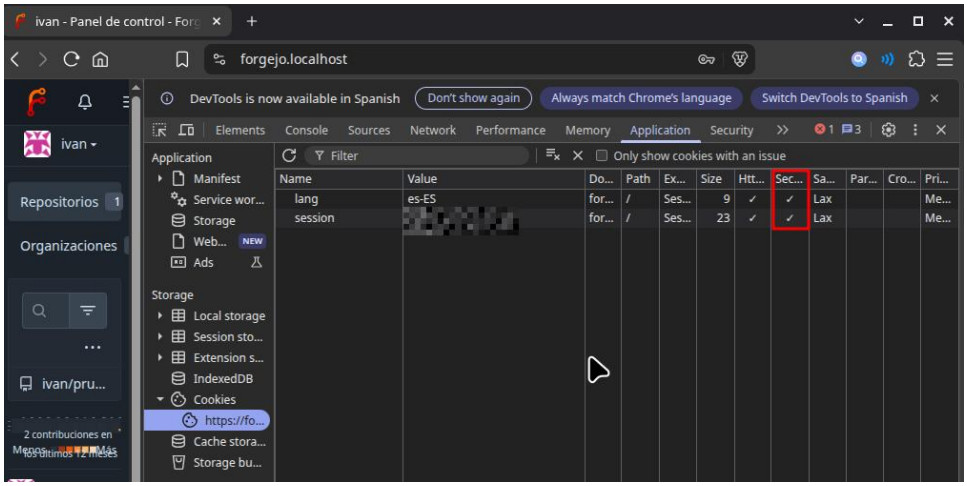
```
❯ git config --global http.https://forgejo.localhost/.sslCAInfo ~/Documentos/2º\ DAW/Despliege\ aplicaiones\ Web/apuntes-daw-2627/d
ocker/practica-http/nginx/certs/ca.crt
ivan@ivan-kubuntu ~/prueba  main
❯ git pull
Ya está actualizado.
```

13. Comprobaciones finales

Comprobamos la redirección HSTS y la cookie

```
ivan@ivan-kubuntu ~/prueba  main
❯ cd ~/Documentos/2º\ DAW/Despliege\ aplicaiones\ Web/apuntes-daw-2627/docker/practica-http/
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -s -o /dev/null -D - http://forgejo.localhost/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.30.6
Date: Sat, 07 Nov 2026 11:51:34 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://forgejo.localhost/

ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -s -o /dev/null -D - --cacert nginx/certs/ca.crt https://forgejo.localhost/
HTTP/2 200
server: nginx/1.30.6
date: Sat, 07 Nov 2026 11:51:51 GMT
content-type: text/html; charset=utf-8
cache-control: max-age=0, private, must-revalidate
x-frame-options: SAMEORIGIN
strict-transport-security: max-age=300
```



Vemos el saludo TLS con y sin –servername

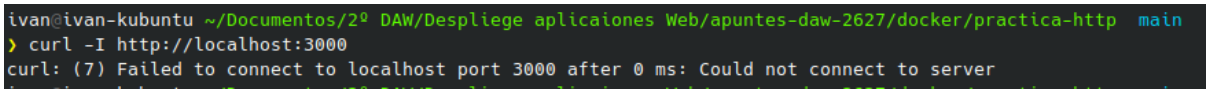
```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ openssl s_client -connect 127.0.0.1:443 -servername forgejo.localhost -CAfile nginx/certs/ca.crt </dev/null 2>/dev/null | grep -E
 "^(subject|issuer)|Protocol|Verify return"
subject=CN=forgejo.localhost
issuer=CN=CA DAW Ivan
Protocol: TLSv1.3
Verify return code: 0 (ok)
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ openssl s_client -connect 127.0.0.1:443 </dev/null
Connecting to 127.0.0.1
CONNECTED(00000003)
40275023B6730000:error:0A000458:SSL routines:ssl3_read_bytes:tlsv1 unrecognized name:../ssl/record/rec_layer_s3.c:918:SSL alert num
ber 112
---
no peer certificate available
---
No client certificate CA names sent
Negotiated TLS1.3 group: <NULL>
---
SSL handshake has read 7 bytes and written 1533 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
Protocol: TLSv1.3
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
```

14. Demostración final

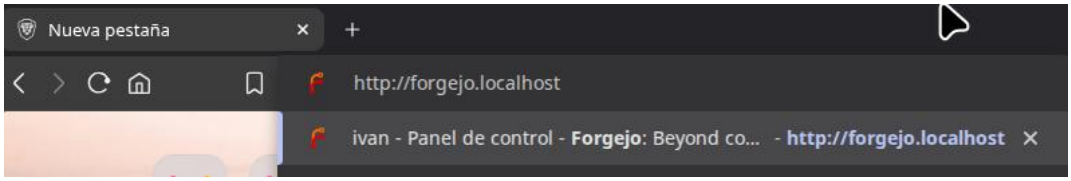
- Contenedores. docker compose ps muestra db, forgejo y nginx en marcha, y solo nginx tiene puertos publicados (80 y 443).

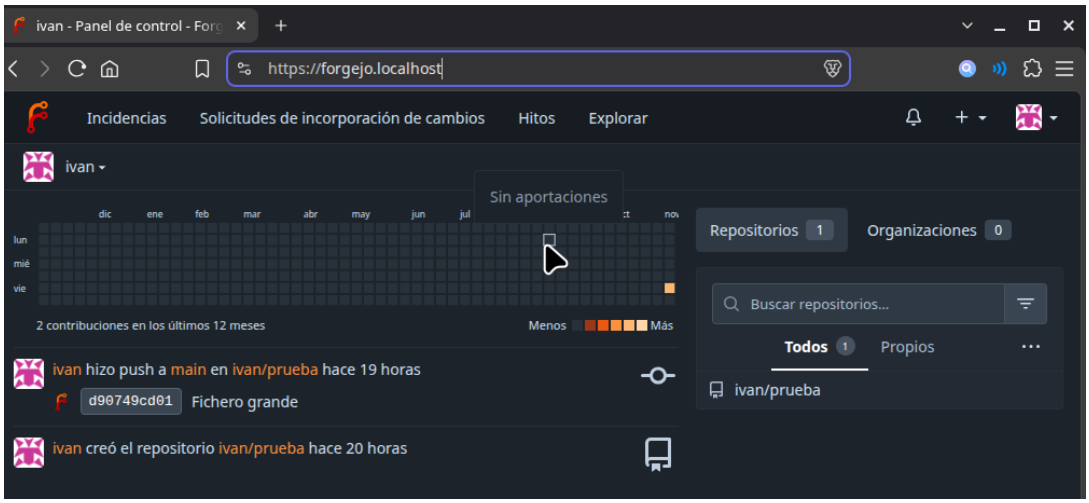


- Forgejo no es accesible directamente. curl -I http://localhost:3000 falla.



- Redirección y candado. Escribe http://forgejo.localhost en el navegador: acaba en https:// con el candado y sin avisos.





- Segundo sitio. https://pages.localhost muestra tu página con tu nombre.



- Nombre desconocido. curl -v -H "Host: hola.localhost" http://127.0.0.1/ termina en Empty reply from server.

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -v -H "Host: hola.localhost" http://127.0.0.1/
*   Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 59330
* using HTTP/1.x
> GET / HTTP/1.1
> Host: hola.localhost
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* Empty reply from server
* shutting down connection #0
curl: (52) Empty reply from server
```

- Certificado con SAN. openssl x509 -in nginx/certs/daw.crt -noout -text | grep -A1 "Alternative Name" muestra los dos nombres, y el certificado lo emite tu CA.

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ openssl x509 -in nginx/certs/daw.crt -noout -text | grep -A1 "Alternative Name"
            X509v3 Subject Alternative Name:
                DNS:forgejo.localhost, DNS:pages.localhost
```

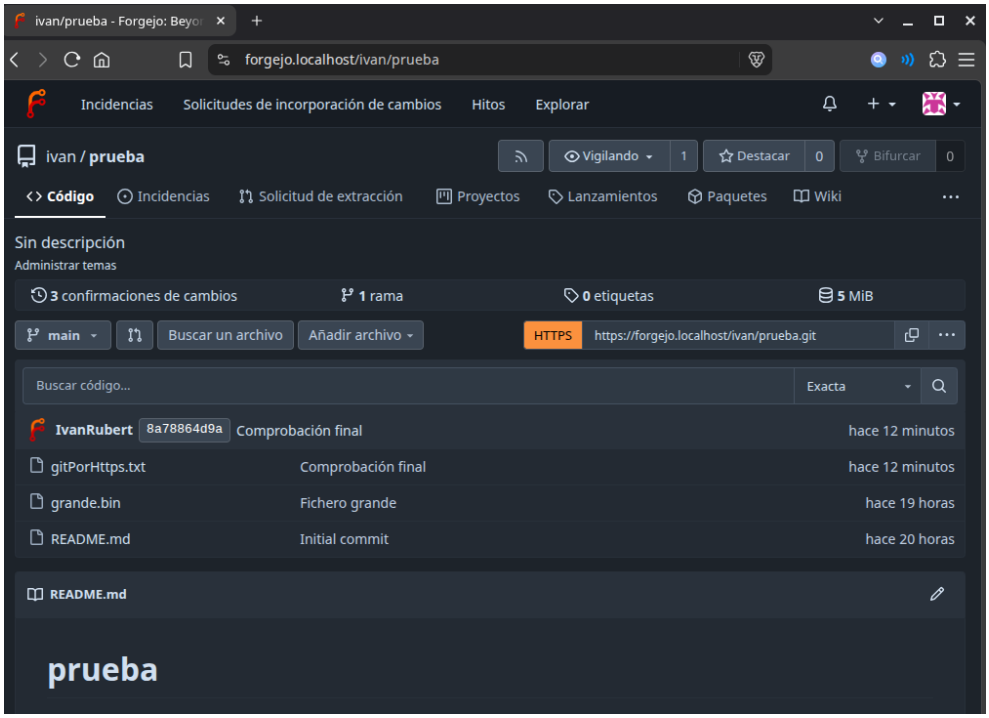
- HSTS. curl -s -o /dev/null -D - --cacert nginx/certs/ca.crt https://forgejo.localhost/ muestra strict-transport-security.

```
ivan@ivan-kubuntu ~/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http  main
❯ curl -s -o /dev/null -D - --cacert nginx/certs/ca.crt https://forgejo.localhost/
HTTP/2 200
server: nginx/1.30.6
date: Sat, 07 Nov 2026 12:44:07 GMT
content-type: text/html; charset=utf-8
cache-control: max-age=0, private, must-revalidate
x-frame-options: SAMEORIGIN
strict-transport-security: max-age=300
```

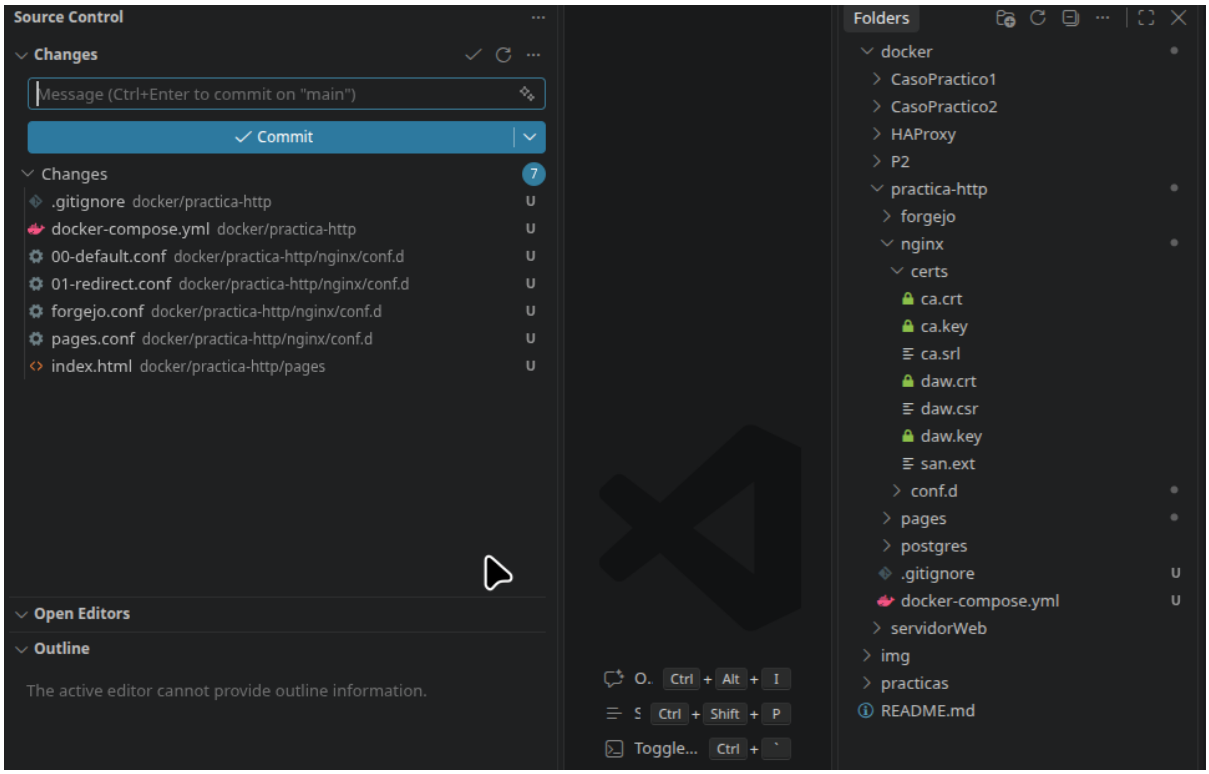
- Git por HTTPS sin trampas. Haz un cambio en ~/prueba, commit y push delante del profesor. git config --global --get http.sslVerify no devuelve nada.

```
ivan@ivan-kubuntu ~/prueba  main
❯ echo "Comprobacion git por HTTPS" > gitPorHttps.txt
ivan@ivan-kubuntu ~/prueba  main
❯ ls
README.md  gitPorHttps.txt  grande.bin
ivan@ivan-kubuntu ~/prueba  main
❯ git add gitPorHttps.txt
ivan@ivan-kubuntu ~/prueba  main●
❯ git commit -m "Comprobación final"
[main 8a78864] Comprobación final
 1 file changed, 1 insertion(+)
 create mode 100644 gitPorHttps.txt
ivan@ivan-kubuntu ~/prueba  main
❯ git config list --global
user.name=IvanRubert
user.email=ivanrubert23@gmail.com
http.https://forgejo.localhost/.sslcainfo=/home/ivan/Documentos/2º DAW/Despliege aplicaiones Web/apuntes-daw-2627/docker/practica-http/nginx/certs/ca.crt
ivan@ivan-kubuntu ~/prueba  main
❯ git push
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (3/3), 355 byte | 355.00 KiB/s, listo.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://forgejo.localhost/ivan/prueba.git
   d90749c..8a78864  main -> main
```

Forgejo detras de Nginx



- Sin claves en el repositorio. En tu repositorio practica-http, git ls-files | grep key no devuelve nada.



- Explícalo. El profesor te hará una pregunta de las que aparecen en «Qué está pasando».
    - Los nombres de tu servidor

        - Al añadir delante de localhost cualquier nombre actúa como un subdominio de ese dominio que apunta a nuestro equipo local por lo que forgejo.localhost y pages.localhost apuntan al mismo equipo pero en los siguientes puntos se distinguen con el proxy inverso para que apunten a un sitio u otro
    - Escribir el docker-compose.yml
        - Se están configurando 2 servicios para desplegarse en contenedores con unas imágenes especificas, volúmenes específicos, puertos a vincular y variables de entorno para preparar su configuración en el despliegue
    - Instalar Forgejo
        - Si la creación de un cuenta administrador de Forgejo por el navegador falla se debe ejecutar este comando para crear con un usuario que no sea root una cuenta de Forgejo, en este caso con el usuario con id 1000 se crea una cuenta administrador con nombre de usuario, contraseña y email personalizado y aplicando una opción para que no deba cambiar la contraseña al iniciar sesión
    - Configurar el sitio de Forgejo en Nginx
        - Se crea un archivo de configuración para que el proxy inverso al usar el dominio forgejo.localhost por el puerto 80 redirija a la ubicación del contenedor de forgejo con el puerto 3000 sin necesidad de exponer ese puerto
    - Añadir Nginx al compose
        - Se añade a docker-compose.yml un campo para desplegar un contenedor Nginx exponiendo el puerto 80 y haciendo un bind mount de la carpeta donde se han configurado los archivos de configuración a la ruta donde se deben colocar esos archivos manteniéndolo de solo lectura para así modificarlo solo en el equipo anfitrión
    - Cerrar el puerto 3000
        - Como ya se ha desplegado el proxy inverso ya no es necesario que se exponga el puerto de forgejo a internet, manteniendo así una conexión mas limpia y segura
    - Fallo provocado: la URL de clonado

        - Nginx reenvía las peticiones, pero Forgejo no sabe con qué dirección le ve el usuario. Para los enlaces absolutos (clonado, correos, webhooks) usa su URL pública, ROOT_URL, que sigue diciendo localhost:3000. Cualquier aplicación detrás de un proxy necesita saber su URL pública.
    - Fallo provocado: el error 413
        - El error 413 (Request Entity Too Large) es debido a que Nginx no permite las peticiones con un body superior a 1MB por defecto por lo que se debe modificar el archivo de configuración para permitir la petición de ese tamaño
    - El sitio en Nginx
        - El archivo de configuración escucha peticiones por el puerto 80 y comprueba si el dominio es pages.localhost, si es muestra el archivo index.html de la carpeta root y si no lo encuentra devuelve un 404
    - Quien decide es la cabecera Host
        - Ambos acceden a 127.0.0.1, pero buscan el resultado que habría salido si se hubiera accedido por el contenido de Host
    - Fallo provocado: un nombre que no existe
        - Por defecto Nginx si no existe el dominio busca el primer dominio que coincida con el puerto y muestra ese
    - Default_server
        - Para evitar lo ocurrido en el punto anterior se debe crear un archivo de configuración por defecto con default_server para que devuelva lo que queremos (en este caso un código 444)
    - Ver solo las cabeceras

      curl -s -o /dev/null -D - http://forgejo.localhost/
        - La opción “-o /dev/null” hace que el cuerpo de la respuesta se descarte y el “-D -” imprime las cabeceras
    - Provocar cada codigo
        - Redirección: Forgejo genera un código de redireccion (3xx)
        - No autenticado: Forgejo genera un código de error 401 Unauthorized con un cuerpo JSON para rutas donde no se tiene permiso

        - No encontrado: Forgejo genera un código 404 cuando se intenta acceder a un recurso que no existe
        - Aplicación caída: Nginx genera un código de error en el servidor (5xx) cuando no se puede acceder al contenedor de Forgejo
    - El inicio de sesión visto por dentro
        - El formulario de inicio de sesión envia una petición POST y cuando recibe una respuesta de que se ha logueado correctamente redirige al usuario con un código 303 y deja la cookie de sesión en el almacenamiento del navegador para mantener el login entre cambios de pantallas
    - El log de acceso
        - Con ese comando se ven los logs del contenedor de Nginx y con el operador "--tail 5" solo muestra los 5 últimos. Cada línea es una petición a Nginx
    - Crear la CA
        - Este comando openssl genera una clave privada (ca.key) y un certificado (ca.crt) valido para los siguientes 5 años desde la creación de ambos archivos y los deja sin contraseña con la opción -noenc
    - Pedir un certificado para el servidor
        - Con este comando se genera una clave para el servidor (daw.key) y una petición de firma (daw.csr) para crear el certificado firmado con la firma del CA
    - Indicar los nombres SAN
        - El campo CN del paso anterior solo admite un nombre, y los navegadores dejaron de mirarlo hace años. Los nombres válidos de un certificado van en la extensión SAN (subjectAltName), que admite varios. Las otras líneas dicen que no es una CA y que sirve para un servidor web.
    - Publicar el 443 y montar el certificado
        - En el docker-compose.yml en el contenedor de Nginx se vinculan los archivos de certificación creados para el servidor (su clave privada "daw.key" y su certificado "daw.crt") como solo

lectura y se vinculan el puerto 443 del contenedor con el 443 del equipo anfitrion.

- Pasar los sitios a HTTPS
  - Para conseguir eso se deben crear los archivos de configuración escuchando el puerto 443 con el dominio y los certificados que usa el servidor y hacer una redirección de los dominios HTTP. Aparte en los archivos de configuración HTTPS se activa HTTP/2 y se configura HSTS para que el navegador no intente nunca HTTP con este sitio. Y por último, "ssl_reject_handshake" rechaza las conexiones TLS a nombres que no son los configurados.
- Que el navegador confie en tu CA
  - Los navegadores no confían en los certificados de los servidores sino en quien lo firma, pero como nosotros lo autofirmamos no confía y por eso sale ese mensaje, pero se puede añadir una CA de confianza en la configuración del navegador para que no salga este mensaje
- Fallo provocado: Git no confía en tu CA
  - Git no usa el almacén del navegador, usa el suyo por lo que si se quiere hacer lo mismo se puede aceptar cualquier CA (lo cual no es recomendable y es muy peligroso) o permitir nuestra CA (que es lo mas recomendable en este caso)
- Comprobar la redirección, HSTS y la cookie
  - La redirección 301 deja un hueco: la primera petición sigue saliendo por HTTP sin cifrar y alguien en medio podría interceptarla. Con HSTS, el navegador recuerda que este sitio solo va por HTTPS y ni siquiera intenta HTTP. Usamos 300 segundos porque es un dominio de pruebas: en producción se pone un año
- Ver el saludo TLS
  - En HTTPS, el servidor tiene que enseñar su certificado antes de recibir la petición cifrada con la cabecera Host. Para saber cuál enseñar, el cliente manda el nombre en el primer mensaje del saludo TLS: es SNI (Server Name Indication). -servername lo envía. Sin él, tu servidor por defecto rechaza la conexión

15. Bibliografía y otras fuentes de información consultadas

- https://pedrosegarra.github.io/DAW/